Python Data Types -Functions/Methods

📁 1. LISTS — "The Journey of the Treasure Hunters"

Core List Methods/Functions

Function / Method Name	Purpose	Syntax
append()	Add to end	list.append(item)
insert()	Add at index	list.insert(index, item)
remove()	Remove first occurrence	list.remove(item)
pop()	Remove last or by index	list.pop() / list.pop(i)
index()	Get index of value	list.index(item)
count()	Count occurrences	list.count(item)
sort()	Sort ascending	list.sort()
reverse()	Reverse list	list.reverse()
copy()	Copy list	list.copy()
extend()	Add elements from another list	list.extend(other_list)
len()	Length of list	len(list)
sum()	Sum of numeric values	sum(list)
max() / min()	Largest / Smallest	max(list) / min(list)

🧩 **List Challenge — “The Journey of the Treasure Hunters”**

A group of treasure hunters track rare artifacts hidden across distant lands. They maintain a list of values collected on their journey. You're the team's scribe — your task is to manage this list using Python.

artifacts = [12, 7, 99, 23, 7, 45, 99, 8, 4, 0]

🧞‍♂️ **Your Tasks:**

1. **A new artifact with value 88 was found — add it to the end.**
2. **A secret item of value 33 must be placed at index 2.**
3. **There’s a rumor that the value 7 is cursed — remove the first occurrence.**
4. **A traitor stole the last artifact — remove it without using the index.**
5. **Another traitor took the first one — remove that one using its index.**
6. **A new legend was found — what's the index of artifact 45?**
7. **The value 99 appears suspiciously often — how many times?**
8. **Sort the artifacts from smallest to largest.**
9. **Reverse the list — the elder wants it in backward order.**
10. **Extract just the 3rd and 4th artifacts using slicing.**
11. **Extract every 2nd artifact from the list.**
12. **Make a copy of the current list into a new variable.**
13. **Merge this new list with another batch: [101, 202, 303].**
14. **Add the sum of all current artifact values to the list.**
15. **Add another entry: the maximum value in the list.**
16. **What is the final length of the list?**
17. **What is the index of 0?**
18. **Find the maximum and minimum artifact values.**
19. **Clear the copy list — the new batch was a decoy!**
20. **Check if the number 23 is still in the original list.**

2. TUPLES — "The Scroll of Ancient Coordinates"

Core Tuple Methods/Functions

Function / Method Name	Purpose	Syntax
len()	Number of items	len(tuple)
index()	Index of value	tuple.index(value)
count()	Count occurrences	tuple.count(value)
in	Membership test	value in tuple
tuple()	Type conversion	tuple(iterable)
Slicing / unpacking	Access sub-values	tuple[start:end], a, b = tuple[:2]

🧩 **Tuple Challenge — "The Scroll of Ancient Coordinates"**

Deep in the archives of the Lost Library, a scroll was found. It holds a set of sacred coordinates stored in a tuple. You, the royal decoder, must interpret this immutable treasure.

coordinates = (5, 8, 12, 5, 42, 7, 8, 0, 99, 5)

📜 **Your Royal Tasks:**

1. **Determine how many coordinates the scroll contains.**
2. **What is the first coordinate? The last one?**
3. **Slice out the first five coordinates.**
4. **Extract only the even-positioned coordinates (0-based index).**
5. **How many times does the number 5 appear?**
6. **Find the position (index) of the first occurrence of 42.**
7. **Unpack the first three coordinates into x, y, z and print them.**
8. **Use slicing and sum() to get the total of the middle five values.**
9. **Convert the tuple into a list, and append the value 100.**
10. **Sort the converted list in descending order.**
11. **Convert it back to a tuple.**
12. **Is the number 12 present in the original tuple?**
13. **Count how many unique values are in the coordinates.**
14. **Create a tuple that is the reverse of the original.**
15. **Join two tuples: this one and (1, 2, 3) into a new tuple.**
16. **Repeat the coordinate tuple twice using repetition operator.**
17. **Slice and extract coordinates from index 3 to 7.**
18. **What is the minimum and maximum coordinate?**
19. **Create a nested tuple: group first 3, next 3, and rest into sub-tuples.**
20. **Write a function that takes a tuple and returns only coordinates > 10.**

3. DICTIONARIES — "The Inventory of the Wizard"

Core Dictionary Methods/Functions

Function / Method Name	Purpose	Syntax
get()	Safe key access	dict.get(key)
keys() / values()	All keys / values	dict.keys() / dict.values()
items()	All key-value pairs	dict.items()
update()	Add/update from another dict	dict.update(other_dict)
pop() / popitem()	Remove by key / last pair	dict.pop(key) / dict.popitem()
clear()	Empty dictionary	dict.clear()
in	Key check	key in dict
len()	Size of dictionary	len(dict)

Practice Challenge: "The Inventory of the Wizard"

Great! Let's now do the same for **Python dictionaries** — a **story-based practice task** that covers the most important dictionary methods and operations, in an **indirect, clue-style format** just like the string one.

🧠 **Python Dictionary Methods & Concepts to Cover**

✅ **Commonly Used Dictionary Methods:**

Method/Concept	Description
dict[key]	Access value by key
dict.get(key)	Safe value access
dict.keys()	Returns all keys
dict.values()	Returns all values
dict.items()	Returns all key-value pairs

Method/Concept	Description
dict.update()	Updates with another dict
dict.pop(key)	Removes key and returns value
dict.popitem()	Removes last inserted pair
dict.clear()	Empties the dictionary
key in dict	Checks if key exists
len(dict)	Number of items
max(dict.values())	Max value
min(dict.keys())	Min key
Looping over dict.items()	Iterate over key-value pairs

🧩 Dictionary Challenge — “The Inventory of the Wizard”

A wizard maintains a magical inventory as a Python dictionary. You are his apprentice, and he asks you to help manage it. But he speaks in riddles.

```
inventory = {
  "wand": 3,
  "potion": 5,
  "scroll": 7,
  "elixir": 2,
  "gem": 10
}
```

🧙‍♂️ **Your Tasks:**

1. The wizard needs to know how many types of items he owns.
2. He wants to see only the names of the items.
3. Now he needs just the item counts.
4. Show him both the item names and their counts.
5. A thief stole the **elixir** — remove it.
6. He wants to know how many **scrolls** he has, but carefully — don’t risk an error if it's missing.
7. A new shipment arrives: {"crystal": 4, "wand": 2} — merge it into the inventory.
8. He thinks "wand" should now be 10 — set it directly.
9. Show whether the "dagger" exists in the inventory.
10. Show whether the "gem" is still safe.
11. The wizard wants to reward you — print the **most valuable item** (highest count).
12. Also, show the item with the **lowest quantity**.
13. He drops the last item added — remove it (even if you don’t know its key).
14. Sort and show items by name (ascending).
15. Sort and show items by value (descending).
16. Loop through the inventory and print each item like: "wand: 3 uses".
17. Create a new dictionary where each quantity is doubled (without changing the original).
18. Sum up the total number of items the wizard owns.
19. The wizard forgets everything — clear the inventory.
20. But wait! He had a backup — restore it from a saved copy.

🔣 4. STRINGS — "The Tale of the Wise Coder"

Core String Methods/Functions

Function / Method Name	Purpose	Syntax
strip()	Remove surrounding spaces	str.strip()
replace()	Replace substrings	str.replace(old, new)
lower() / upper()	Case transformation	str.lower() / str.upper()
title()	Title case	str.title()
count()	Count occurrences	str.count(sub)
find() / index()	Find substring index	str.find(sub) / str.index(sub)
split() / join()	Convert to list or back	str.split(sep) / 'sep'.join(list)
startswith() / endswith()	Prefix/suffix check	str.startswith(sub) / str.endswith(sub)
in	Substring presence	substr in str

5. SETS — "The Brotherhood of Unique Blades"

Core Set Methods/Functions

Function / Method Name	Purpose	Syntax
add()	Add a new element	set.add(value)
remove() / discard()	Remove element safely	set.remove(x) / set.discard(x)
union()	Combine sets	set1.union(set2)
intersection()	Common elements	set1.intersection(set2)
difference()	Elements in set1 not set2	set1.difference(set2)
symmetric_difference()	Elements in only one	set1.symmetric_difference(set2)
copy()	Create copy	set.copy()
clear()	Empty the set	set.clear()
in	Membership test	value in set
issubset() / issuperset()	Relation test	set1.issubset(set2)
Isdisjointset()	Check both have common are not	Set1.disjoint(set2)

🧩 **Set Challenge — "The Brotherhood of Unique Blades"**

The Brotherhood guards a sacred vault of **unique magical blades**, each blade identified by a number. No duplicate blade can exist. As the newly appointed Vault Master, you must maintain and analyze the Brotherhood's set.

blades = {101, 204, 101, 305, 408, 204, 509, 612, 408}

🧙‍♂️ **Your Vault-Keeper Tasks:**

1. Check how many **unique blades** are in the vault.
2. Add a new blade 713 to the collection.
3. Attempt to add 305 again — what happens?
4. Remove blade 509 from the vault.
5. Safely try removing blade 888 without causing an error.
6. Create another set: new_findings = {305, 888, 999}
7. Merge the two vaults into one (union).
8. Find the blades common to both vaults (intersection).
9. Identify the blades **only** in the original set but not in new_findings.
10. Check if blade 204 is present in the vault.
11. Convert the set to a list and sort it ascending.
12. Find the **largest** and **smallest** blade numbers.
13. Empty the set completely — the vault has been stolen!
14. Restore it from backup: {101, 204, 305, 408, 509, 612}
15. Check if the vault is now a **subset** of another archive: {101, 204, 305, 408, 509, 612, 713, 888}
16. Find if the archive is a **superset** of the vault.
17. Find the **symmetric difference** (blades that are in one but not both).
18. Create a copy of the vault — you'll need it in the future.
19. Use a loop to print: "Blade #101 secured." for each blade.
20. Write a function that takes any set and returns only **even-numbered blades**.